

PES UNIVERSITY, BENGALURU

Department of Computer Science and Engineering

Software Architecture and Design Specification

Online Bookstore

Version 1.0 — Draft for Team Review

Based on Online Bookstore Software Requirements Specification, Version 1.0 — Draft

Prepared by: Online Bookstore Project Team

Organization: PES University

Date: October 2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	October 2026	Project team	Initial SAD draft based on the supplied SRS and SAD template.

Approvals

Role	Name	Signature / Date
Project representative	To be completed	Pending
Instructor / guide	To be completed	Pending

Document status: Draft for review. Proposed design choices are identified as such; they should not be treated as approved decisions until the team confirms them.

Contents

Revision History.....	2
Approvals.....	2
1 Introduction.....	4
1.1 Purpose.....	4
1.2 Scope.....	4
1.3 Audience.....	4
1.4 Definitions.....	4
1.5 Related Documents.....	4
2 Document Overview.....	5
2.1 How to Use This Document.....	5
2.2 Related Design Status.....	5
3 Architecture.....	5
3.1 Goals and Constraints.....	5
3.2 Stakeholders and Concerns.....	5
3.3 Component (UML) Diagram.....	6
3.4 Component Descriptions.....	7
3.5 Chosen Architecture Pattern and Rationale.....	7
3.6 Technology Stack and Data Stores.....	8
3.7 Risks and Mitigations.....	8
3.8 Traceability to Requirements.....	8
3.9 Security Architecture.....	9
4 Design.....	10
4.1 Design Overview.....	10
4.2 UML Sequence Diagrams.....	10
4.2.1 Registration and Login.....	10
4.2.2 Add a Book to Cart.....	11
4.2.3 Place an Order.....	12
4.3 API Design.....	13
4.4 Error Handling, Logging and Monitoring.....	14
4.5 UX Design.....	14
4.6 Open Issues and Next Steps.....	14
5 Appendices.....	15
5.1 Glossary.....	15
5.2 References.....	15
5.3 Tools.....	15
Document Review Checklist.....	15

1. Introduction

1.1 Purpose

This Software Architecture and Design (SAD) document describes the proposed architecture, components, data model, interactions, interfaces, security controls, and verification approach for the Online Bookstore. It translates the requirements in the Online Bookstore Software Requirements Specification (SRS) into a design that the development and testing team can review.

This document is a design baseline, not evidence that the system has already been implemented. Items identified as proposed or to be confirmed remain subject to team approval.

1.2 Scope

The design covers user registration and login; book browsing and searching; seller book-listing creation; customer cart add/remove operations; order placement; order-history viewing; role-based access control; and protection of user data.

Payment processing, shipping and tracking, reviews, ratings, recommendations, discounts, inventory replenishment, seller payouts, and administrator dashboards are not specified in the supplied SRS and are outside this version unless formally approved.

1.3 Audience

- Developers implementing the Django application, models, views, and templates.
- Testers preparing requirement-based, integration, authorization, and regression tests.
- Project guide or reviewer assessing architecture quality and requirement coverage.
- Team members maintaining the design as implementation decisions are finalized.

1.4 Definitions

Term	Definition
SAD	Software Architecture and Design.
SRS	Software Requirements Specification.
Django ORM	Django's model-based interface for database operations.
RBAC	Role-Based Access Control; restricting operations according to a user's assigned role.
PK / FK	Primary key / foreign key.
CSRF	Cross-Site Request Forgery.
XSS	Cross-Site Scripting.
Cart item	A selected book and quantity associated with a customer.
Order item	A line in an order containing a book, quantity, and purchase-time unit price.

1.5 Related Documents

- Online Bookstore Software Requirements Specification, Version 1.0 — Draft, October 2026.
- Software Architecture and Design Specification template supplied for this project.
- Final approved use-case diagram, test plan, and project guidelines when available.

2. Document Overview

2.1 How to Use This Document

Section 3 describes the architecture, components, technology options, data model, risks, traceability, and security controls. Section 4 describes implementation-level design, key workflows, route/interface behavior, validation, error handling, user experience, and testing. Section 5 provides appendices.

2.2 Related Design Status

The supplied SRS identifies Django as the required framework and describes a database-backed web application with Customer and Seller roles. It does not finalize the database engine, deployment environment, or the precise role-assignment rule at registration. These decisions are left open below rather than assumed to be approved.

Decision	Current status	Design treatment
Application framework	Specified by SRS	Python Django is the required framework.
Database engine	Not specified by SRS	Use a relational database through Django ORM; select the engine with the team.
Deployment environment	Not specified by SRS	Describe a logical browser–Django–database deployment; hosting remains open.
Registration role selection	Needs explicit confirmation in SRS	Design supports Customer or Seller role assignment; confirm whether users select the role at registration and whether it can later change.
Security objectives marked TBD	Pending SRS update	Proposed controls are documented, then must be reconciled with final security requirements.

3. Architecture

3.1 Goals and Constraints

Goals

- Cover the functional scope described in the SRS.
- Separate authentication, book management, cart, and order responsibilities.
- Maintain consistent cart and order data after successful operations.
- Support the SRS book-search response-time target of under three seconds under normal conditions.
- Support the latest stable versions of the specified browsers.
- Keep functional requirements independently testable.

Constraints

- The application must use Python Django and operate as a web-based application.
- The system must persist user accounts, roles, book listings, cart data, and orders.
- Customer and Seller are the two primary roles identified in the SRS.
- Protected operations must enforce role restrictions on the server.
- The SRS availability target is at least 99% of scheduled operating time, excluding planned maintenance; actual achievement depends on the selected hosting and operations.
- The database engine and deployment configuration are not finalized by the SRS.

3.2 Stakeholders and Concerns

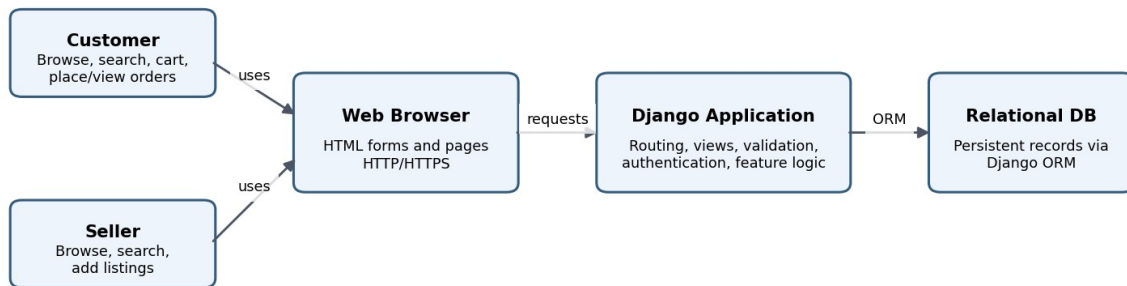
Stakeholder	Primary concerns	Architectural response
-------------	------------------	------------------------

Customer	Simple browsing/search, accurate cart and orders, privacy.	Clear pages, validation, and queries scoped to the logged-in customer.
Seller	Correct and straightforward book listing creation.	Seller-only listing operation with server-side role checks.
Developers	Modularity, maintainability, ease of change.	Feature modules and separation between views, business logic, and data models.
Testers	Verifiable requirements and repeatable tests.	Traceability, explicit validation rules, and testable modules.
Project guide/reviewer	Consistency with SRS and template.	Architecture diagrams, rationale, traceability, and open-issue tracking.
Deployment operator	Availability, secrets, recovery, and safe configuration.	Environment-specific configuration, backups, and operational checks.

3.3 Component (UML) Diagram

The system is designed as a browser-based application in which Django receives requests, applies authentication and authorization, validates input, coordinates feature logic, and accesses persistent data through the Django ORM.

Online Bookstore — High-Level Architecture



Database engine and deployment platform are not fixed by the SRS.

Figure 1. High-level architecture and request/data flow.

Django Application — Logical Components

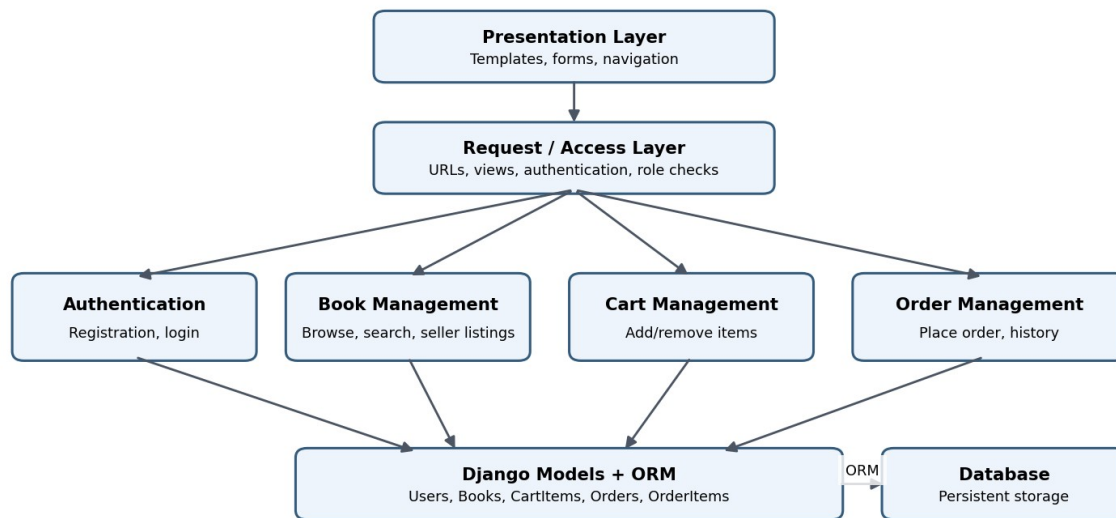


Figure 2. Logical component diagram for the Online Bookstore.

3.4 Component Descriptions

Component	Responsibility
Browser / User Interface	Displays registration, login, catalogue, search, cart, order, and seller listing pages; submits user actions.
URL Routing and Views	Routes requests to handlers; checks login and role; validates input; coordinates feature operations; returns pages or redirects.
Accounts / Authentication	Uses Django authentication facilities for account creation, login/logout, sessions, and role assignment.
Book Catalogue	Retrieves book listings, supports browsing/search, and creates listings for authorized sellers.
Cart Management	Adds, displays, and removes cart items; validates quantities and associates cart records with the correct customer.
Order Management	Validates the cart, calculates totals from server-side book data, creates orders and order items, and retrieves order history.
Django Models and ORM	Represents persistent entities and performs database operations through Django's data-access layer.
Relational Database	Stores user, role, book, cart, and order records with appropriate constraints and relationships.

3.5 Chosen Architecture Pattern and Rationale

Proposed pattern: a layered modular monolith implemented with Django.

- Presentation layer: templates, forms, and static assets render pages and collect input.
- Request and access layer: URL routing and views authenticate requests, enforce permissions, validate data, and coordinate operations.
- Feature layer: focused modules or service functions implement book, cart, and order rules.
- Data-access layer: Django models and ORM encapsulate database interaction.
- Persistence layer: a relational database stores application records.

This pattern suits the project's bounded scope and mandatory Django framework. It keeps deployment and debugging relatively simple while preserving clear module responsibilities. A microservices architecture is not recommended for this version because the supplied requirements do not justify the additional service and deployment complexity.

3.6 Technology Stack and Data Stores

Area	Technology / approach	Status or rationale
Web framework	Python Django	Required by the SRS.
Data access	Django ORM and Django models	Proposed implementation approach consistent with Django.
Database	Relational database (PostgreSQL, MySQL, or SQLite are possible options)	Engine not specified by the SRS; confirm with the team. PostgreSQL is a candidate, not an approved deployment decision.
User interface	Django templates, HTML, CSS, optional JavaScript	Proposed browser interface; final UI details may evolve.
Authentication	Django authentication and password hashing	Use framework-supported password and session handling.
Transport security	HTTPS in hosted environments	Depends on deployment configuration; use secure cookie and secret settings where applicable.
Version control	Git	Recommended for collaborative development and review.

The logical architecture does not require the team to choose a hosting provider at this stage. During development, the application can run locally. For a demonstration or production-like deployment, it may be hosted on a managed application platform or a virtual server; Docker may be used to package the application. The selected option should be documented after the team's deployment needs are known.

3.7 Risks and Mitigations

Risk	Potential impact	Mitigation
Database engine or schema chosen late	Migrations or environment setup may need rework.	Confirm the engine before implementation becomes dependent on engine-specific features; keep models portable where practical.
Incorrect role enforcement	Unauthorized listing or purchasing actions.	Check roles on the server for every protected view and test wrong-role access.
Concurrent cart/order changes	Incorrect totals or inconsistent records.	Validate current book data on the server and create related order records within a database transaction.
Database or host outage	Users cannot access the application or persistent data.	Select hosting appropriate to the availability target; document backups and recovery procedures.
Sensitive information in logs or repository	Privacy or credential exposure.	Do not log passwords, session identifiers, or secrets; keep deployment secrets out of source control.
Search performance below target	Search takes longer than the SRS target.	Measure response time, bound search queries, and add appropriate indexes if evidence supports them.

3.8 Traceability to Requirements

The following matrix maps the SRS requirement areas to the proposed design. Requirement identifiers should be checked against the final SRS and expanded to exact IDs if the SRS changes.

SRS requirement area	Design components / sections	Verification focus
----------------------	------------------------------	--------------------

Registration and login (FR)	Accounts; Section 4.2.1	Valid/invalid registration, login, logout, and session behavior.
Browse and search books (FR)	Book Catalogue; Section 4.3	Correct results, empty results, search latency target.
Seller adds book listings (FR)	Book Catalogue; Security Architecture	Seller can add; unauthenticated users and Customers are denied.
Add/remove cart items (FR)	Cart Management; Section 4.2.2	Valid item/quantity, removal, ownership, and invalid input.
Place order (FR)	Order Management; Section 4.2.3	Valid cart, server-calculated total, transaction consistency.
View order history (FR)	Order Management; Section 4.3	Only the logged-in customer's orders are returned.
Role-based access (SEC / business rules)	Accounts, views, server-side permission checks	Wrong-role requests are denied without modifying data.
Protect user data (SEC)	Security Architecture; Error Handling	Safe password/session handling, CSRF protection, no sensitive logs.
Performance, availability, compatibility, maintainability, testability (NFR)	Architecture, deployment options, tests	Measure against the targets and record the test environment/results.

3.9 Security Architecture

Security is enforced at the server boundary and data layer. Hiding a button in the interface is not sufficient access control; every protected operation must validate the authenticated user's permissions.

Authentication and authorization

- Use Django's authentication and password-hashing mechanisms; never store plaintext passwords.
- Require authentication for cart changes, order placement/history, and seller listing creation.
- Only an authenticated Seller may create book listings; only an authenticated Customer may perform customer purchasing operations, as described in the SRS.
- If users select Customer or Seller at registration, validate the selection and store the assigned role server-side. Confirm with the team whether role changes are allowed later.
- Scope cart and order queries to the logged-in user; do not trust client-supplied customer identifiers.
- Deny actions for users with the wrong role and avoid exposing private records.

Application and data protection

- Use CSRF protection for state-changing browser forms.
- Use Django template escaping and avoid unsafe rendering of user-supplied book descriptions.
- Validate input and use ORM queries rather than constructing SQL strings from user input.
- Use HTTPS, secure cookie settings, and production-safe settings in a hosted environment.
- Keep secret keys and database credentials out of source control; restrict access to the database and backups.
- Do not log passwords, session identifiers, secret keys, or unnecessary personal data.
- Display safe user-facing errors and log only non-sensitive diagnostic details.

Threat review

Threat category	Example	Proposed control
Spoofing / account misuse	Attempt to access another account.	Django authentication, session controls, and ownership checks.
Tampering	Submit invalid prices, quantities, or identifiers.	Server-side validation; derive prices from stored book records.
Information disclosure	Read another customer's order history.	Authorization checks and user-scoped queries.
Cross-site request forgery	Submit an unwanted state-changing request.	CSRF tokens for browser form submissions.

Injection / unsafe content	Malicious search input or book description.	ORM parameterization, validation, and template escaping.
Denial of service	Excessive costly searches.	Bounded queries and hosting-level controls where required.
Privilege escalation	Customer invokes seller-only route.	Server-side role enforcement on every protected route.

The SRS contains security-objective items marked TBD. The controls above are proposed design measures; they must be reconciled with the final security objectives and requirements before the SAD is approved.

4. Design

4.1 Design Overview

Django URL routing dispatches requests to focused views. Views check authentication and role, validate forms, call feature logic, and return templates or redirects. Django models represent persistent entities and the ORM handles database access. Operations that modify multiple related records—especially order placement—should use database transactions.

Suggested Django project organization

Module / path	Responsibility
config/ or project package	Settings, root URLs, WSGI/ASGI entry points.
accounts/	Registration, login/logout integration, role assignment, and account forms.
books/	Catalogue browsing/search and seller listing creation.
cart/	Cart item model and add/remove/display operations.
orders/	Order and OrderItem models, order placement, and order history.
templates/ and static/	HTML templates, CSS, images, and JavaScript assets.
tests/	Model, form, view, permissions, workflow, and regression tests.
requirements.txt / environment config	Python dependencies and deployment-specific configuration; no secrets in source.

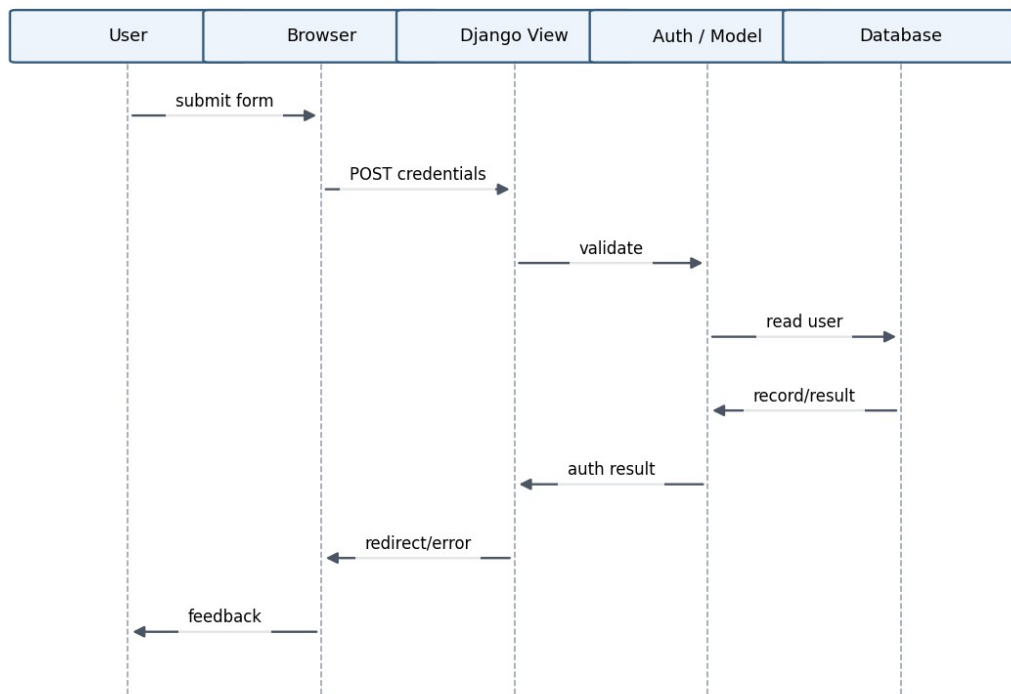
This is a proposed module layout. The team may use one Django app or multiple apps as long as responsibilities and access rules remain clear.

4.2 UML Sequence Diagrams

4.2.1 Registration and Login

- 1.The user opens the registration or login page.
- 2.The browser submits the form to Django.
- 3.Django validates the submitted fields and credentials.
- 4.For registration, the system creates the account only after validation and applies the agreed role-assignment rule.
- 5.For login, Django establishes a session only when the credentials are valid.
- 6.The system redirects to the appropriate page or displays a safe error.

Sequence — Registration / Login



Use Django authentication; never store plaintext passwords.

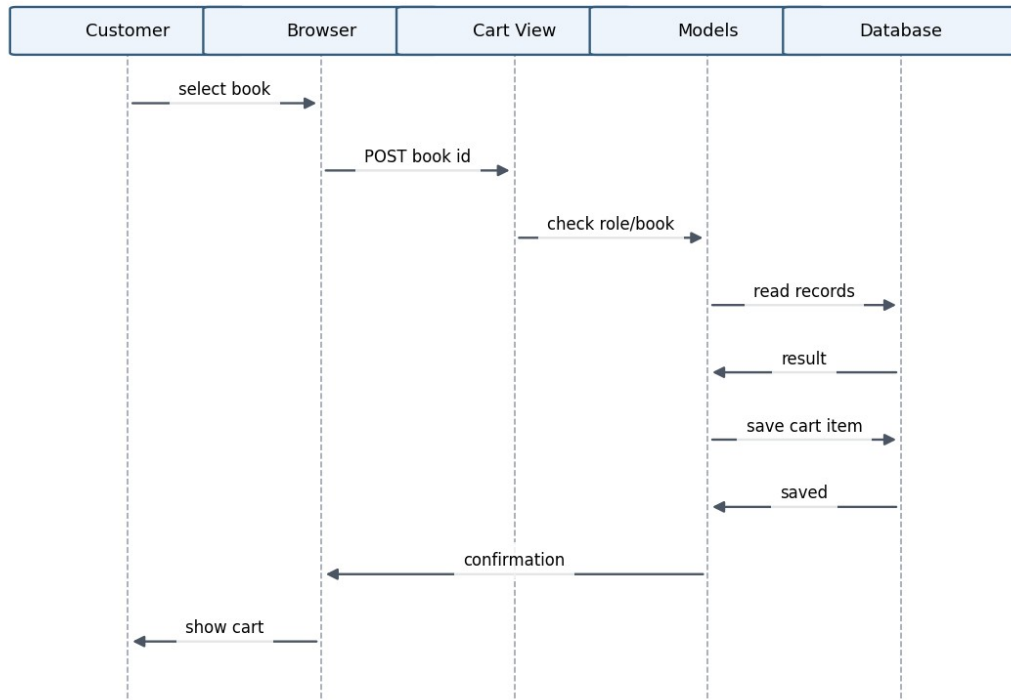
Figure 3. Registration and login interaction sequence.

Open decision: confirm in the SRS whether the user explicitly chooses Customer or Seller during registration. If this is the intended behavior, the role selection must be validated and stored by the server; the role must not be inferred from a client-controlled URL or hidden field alone.

4.2.2 Add a Book to Cart

7. The Customer submits the add-to-cart form.
8. Django checks the authenticated session and Customer role.
9. The application verifies that the book exists and the quantity is valid.
10. Cart logic creates a CartItem or updates an existing line according to the team's duplicate-add rule.
11. The database persists the change and the interface shows the updated cart or a safe error.

Sequence — Add Book to Cart



Derive customer identity from the session, not a client-supplied customer ID.

Figure 4. Add-to-cart interaction sequence.

4.2.3 Place an Order

12. The Customer submits order placement from the cart.
13. Django checks login status, Customer role, and cart contents.
14. The server loads book records and current prices, validates items, and calculates the total.
15. Within one database transaction, the application creates the Order and associated OrderItem records.
16. On success, the transaction commits and the interface shows confirmation.
17. On failure, the transaction rolls back and the interface shows a safe error.

Sequence — Place Order

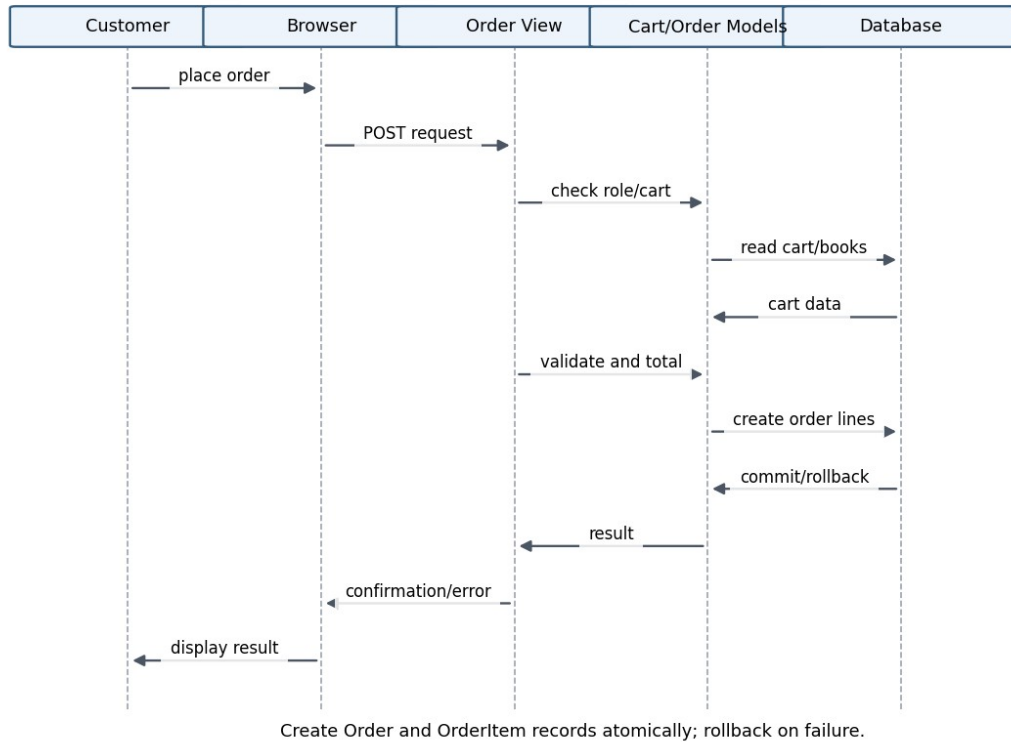


Figure 5. Place-order interaction sequence.

The SRS does not specify payment processing. In this design, placing an order records the order; payment functionality must not be assumed unless it is added to the approved requirements.

4.3 API Design

The SRS requires browser-based interfaces but does not require a public REST API. The following are proposed Django browser routes; the team may adjust the route names to match implementation.

Route	Method	Access	Purpose / behavior
/register/	GET, POST	Public	Display registration form; validate input and create account with approved role rule.
/login/	GET, POST	Public	Display login form and establish a session after valid credentials.
/books/	GET	Public or authenticated	Browse available book listings.
/books/search/	GET	Public or authenticated	Search book listings using bounded, validated query parameters.
/books/add/	GET, POST	Authenticated Seller	Create a book listing after server-side validation.
/cart/	GET	Authenticated Customer	Display the current customer's cart.
/cart/add/	POST	Authenticated Customer	Add or update a cart item after validation.
/cart/remove/<id>/	POST	Authenticated Customer	Remove an item only if it belongs to the current customer's cart.
/orders/place/	POST	Authenticated Customer	Validate the cart and create order records in a transaction.

/orders/	GET	Authenticated Customer	Display only the current customer's order history.
----------	-----	------------------------	--

Interface behavior

- Use GET to display or retrieve information and POST for state-changing actions.
- Use CSRF protection on POST forms.
- Redirect after successful form submission to reduce accidental duplicate submissions.
- Display field-level validation errors without exposing internal details.
- Redirect unauthenticated users to login or deny access as appropriate.
- Deny logged-in users whose role does not permit the requested action.

4.4 Error Handling, Logging and Monitoring

User-facing errors

- Invalid registration: show field errors and do not create an account.
- Invalid login: use a generic authentication failure message.
- Wrong role: deny the action and do not modify data.
- Missing book or invalid quantity: explain the validation issue and leave the cart unchanged.
- Empty or invalid cart: do not create an order; explain the required correction.
- Unexpected or database failure: show a generic message, log safe diagnostics, and never expose production stack traces.

Logging and operational checks

- Log application errors and failed order attempts without secrets or sensitive values.
- Use separate development and hosted-environment settings.
- Monitor application availability and application/database errors in the selected environment.
- Measure search latency against the three-second SRS target.
- Back up persistent data and periodically verify restore procedures.
- Document a basic incident and recovery procedure before deployment.

4.5 UX Design

- Use consistent navigation, labels, page layout, and feedback patterns.
- Make catalogue browsing and search easy to reach.
- Display book title, author, price, and description where available.
- Separate seller listing actions from customer purchasing actions.
- Show cart contents and confirmation after adding or removing items.
- After order placement, show confirmation with the saved order identifier and total where available.
- Show clear empty states for no search matches or no order history.
- Use readable text, labelled controls, keyboard-friendly navigation, and responsive layouts.

4.6 Open Issues and Next Steps

Open item	Action required	When to close
Database engine	Select PostgreSQL, MySQL, SQLite, or another approved relational database; record the rationale.	Before finalizing environment-specific setup and migrations.
Deployment environment	Choose local-only demonstration, managed application hosting, VPS/cloud VM, or containerized deployment as appropriate.	When deployment and demonstration needs are known; not required to define the logical architecture.
Role selection at registration	Confirm that users choose Customer or Seller during registration and whether role changes are allowed later; update the SRS explicitly.	Before implementation of registration and authorization.

Security objectives	Replace TBD security-objective items in the SRS with agreed, testable requirements and align this SAD.	Before design approval.
Final analysis models	Add or reconcile the approved use-case diagram and any required final analysis models.	When the relevant team member completes the diagrams.
Exact test identifiers	Add test case IDs to the traceability matrix when the test plan is available.	During test planning.
Implementation alignment	Review diagrams, schema, routes, and permission checks against the implemented Django modules.	Before final submission/release.

5. Appendices

5.1 Glossary

Term	Definition
Authentication	Verifying a user's identity.
Authorization	Determining whether a user may perform an operation.
Customer	Registered user who browses and purchases books.
Seller	Registered user who adds book listings.
Book listing	Information representing a book available on the platform.
Cart item	A selected book and quantity associated with a customer.
Order	Persistent record of a customer's purchase request.
Order item	A line in an order with a book, quantity, and purchase-time unit price.
Django ORM	Django's model-based interface for database operations.
Layered modular monolith	One deployable application divided into modules/layers with clear responsibilities.
Traceability	Mapping a requirement to design elements and verification activities.

5.2 References

18. PES University, Department of Computer Science and Engineering, Online Bookstore Software Requirements Specification, Version 1.0 — Draft, October 2026.
19. Software Architecture and Design Specification template supplied for the project.
20. Django documentation for authentication, forms, models, database transactions, CSRF protection, deployment checklist, and testing; consult the version selected for implementation.

5.3 Tools

- Microsoft Word or a compatible DOCX editor.
- PlantUML, diagrams.net (draw.io), or equivalent for editable diagrams.
- Django test framework and Python test tools.
- Git for version control and review.

Document Review Checklist

- ☐ Confirm all proposed design choices and open issues.
- ☐ Check the design against the approved SRS and final use-case diagram.
- ☐ Finalize schema, constraints, and migrations.
- ☐ Add test identifiers to the traceability matrix when the test plan exists.

- ☐ Review access and ownership checks on every protected route.
- ☐ Ensure diagrams match implemented modules and models.
- ☐ Record approvals and update revision history.